

VII ADC w Pulse With Modulation

Contreras R Orlando^{1,2*} and Lara H Kevin^{1,2*}

³Department of Electronic Systems, UAA, Av. Universidad 940,
Aguascalientes, 20131, Aguascalientes, México.

*Corresponding author(s). E-mail(s): [al348390,al464376](mailto:al348390@edu.uaa.mx)@edu.uaa.mx;

Abstract

The objective of this lab practice is develop a code able to perform a change in the PWM duty cycle using the samples of an audio signal captured by the ADC module integrated into the STM32F446ZET6 and at the same time, take out the complementary PWM signal an view the change of both PWM on the oscilloscope.

Keywords: ARM, C, ADC, PWM, Samples.

1 Introduction

Implementing hardware solutions in micro controllers is an smart way to improve latency in real time systems, this is like-a-FPGA solution in embedded system. To leverage this in a optimal way, we use multiple hardware solutions like Timers, Counters, ADC, DAC and multiple interruptions.

2 Objectives

- Configure TIM1 for complementary PWM with dead-time.
- Modulate PWM duty by an audio signal (500 Hz–14 kHz).
- Sample with ADC via software (timer ISR sets flag; ADC started & polled in main) understand limits without DMA.
- Store sample buffers (RAM) and analyze pre/post filter.
- Learn tradeoffs: sampling jitter, fs limits without DMA, anti-alias filter necessity.

3 Method

3.1 Pseudo-code

Here is the pseudo-code that represents the algorithm, it is also included the configuration as masks or

Algorithm 1 Timer TIM4 Configuration

Input: None

Output: TIM4 configured

```
1 TIM4.PSC ← 0 // No prescaler
2 TIM4.ARR ← 0xFFFF // 16-bit max period
3 TIM4.CCMR1 ← TIM4.CCMR1 ∨ (1 ≪ 0) // Enable channel 1
4 TIM4.CCER ← TIM4.CCER ∨ ((1 ≪ 0) ∨ (1 ≪ 3)) // Capture enable
5 TIM4.SMCR ← TIM4.SMCR ∨ ((7 ≪ 0) ∨ (5 ≪ 4)) // Trigger TI1FP1, rising
   edge
6 TIM4.CR1 ← TIM4.CR1 ∨ (0x81 ≪ 0) // Auto-reload + counter enable
7 TIM4.DIER ← TIM4.DIER ∨ TIM_DIER_UIE // Overflow interrupt enable
8 TIM4.EGR ← TIM4.EGR ∨ TIM_EGR_UG // Update registers
9 TIM4.CR1 ← TIM4.CR1 ∨ (1 ≪ 0) // Start timer
```

Algorithm 2 TIM2 Interrupt Handler for ADC-driven PWM

Input: ADC1.DR value (12-bit)

Output: TIM1.CCR1 updated

```
10 while TIM2.SR ∧ (1 ≪ 0) do
11   TIM1.CCR1 ← (ADC1.DR) ≪ 4 // PWM duty from ADC sample
12   TIM2.SR ← TIM2.SR ∧ ¬(1 ≪ 0) // Clear UIF flag
```

4 Results and Discussion

4.1 Code

Here is the code that represents the previous pseudo-code

Code: main.c

```
#include "conf.h"
#include <stm32f446xx.h>
int main(void){
    config();
    while(1);
}
void TIM2_IRQHandler(void){
    while (TIM2->SR & (1<<0))
    {
        TIM1->CCR1 = (ADC1->DR)<<4;
        TIM2->SR &= ~(1<<0);
    }
}
```

```
Code: conf.c
```

```
#include "conf.h"
#include <stm32f446xx.h>

void confRCC(void){
    RCC->AHB1ENR|=(1<<0)|(1<<4);
    RCC->APB2ENR|=(1<<1);
}
void confGPIO(void){
    GPIOE.MODER|=(0x5555<<16);
    GPIOA.PUPDR|=(0xA<<10);
}
void confTIMER(void){
    TIM4.PSC=0;
    TIM4.ARR=0xFFFF;
    TIM4.CCMR1|=(1<<0);
    TIM4.CCER|=(1<<0)|(1<<3);
    TIM4.SMCR|=(7<<0)|(5<<4);
    TIM4.CR1|=(0x81<<0);
    TIM4.DIER|=TIM_DIER_UIE;
    TIM4.EGR|=TIM_EGR_UG;
    TIM4.CR1|=(1<<0);
}
void configNVIC(void){
    NVIC_EnableIRQ(TIM4_IRQn);
    NVIC_SetPriority(TIM4_IRQn,1);
}
void config(void){
    confRCC();
    confGPIO();
    confTIMER();
    configNVIC();
}
```

4.2 Wiring Diagram

Here is our circuit implementation; The pinout is the following PA8 → PWM 1/1, PA7 → PWM 1/1 Complementary, PA0 → ADC IN0. [Fig 1]

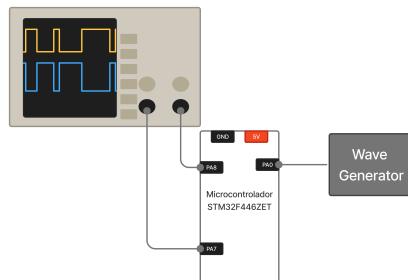


Fig. 1: Wiring Diagram

Q1: What was the maximum f_s you achieved with polling? How did you measure it?

The maximum sampling frequency using polling was limited by the sum of the ADC conversion time, the CPU polling loop time, and any interrupt or bus contention delays. Practically, the maximum f_s achieved was on the order of a few hundred kilohertz.

$$f_s \approx \frac{1}{t_{\text{ADC}} + t_{\text{poll}} + t_{\text{overhead}}}$$

The measurement was done by toggling a GPIO pin around the polling routine, observing the GPIO pulse on an oscilloscope, and computing

$$f_s = \frac{1}{\Delta t_{\text{loop}}}.$$

Q2: What jitter/latency problems appeared and why?

Polling introduced timing uncertainty because the CPU did not always reach the polling instruction at the exact same moment. Interrupt activity, memory wait states, and bus arbitration produced jitter:

$$\Delta t_{\text{jitter}} \neq 0.$$

This caused sampling instants to drift slightly, which results in amplitude errors proportional to input frequency:

$$e_{\text{jitter}}(t) \approx A 2\pi f_{\text{in}} \Delta t_{\text{jitter}}.$$

Latency was also present because the ADC completed the conversion before the CPU read the data, but latency itself is less harmful than jitter.

Q3: How would DMA + hardware triggers change the design?

Using hardware triggering (e.g., a timer triggering the ADC) fixes the sampling instant:

$$t_{\text{sample}} = nT_s \quad \text{with} \quad T_s = \frac{1}{f_s}.$$

DMA removes CPU involvement in data transfer, eliminating polling delays and ISR overhead. The resulting system achieves:

$$\Delta t_{\text{jitter}} \approx 0,$$

and the CPU is free for DSP or control tasks.

Q4: How does dead-time influence waveform symmetry and distortion?

Dead-time introduces a period during which both complementary PWM outputs are off:

$$t_{\text{dead}} > 0.$$

If the dead-time is not perfectly matched or becomes a significant fraction of the switching period, the effective duty cycle changes:

$$D_{\text{eff}} = D_{\text{cmd}} - \frac{t_{\text{dead}}}{T_{\text{PWM}}}.$$

Asymmetry in dead-time between high-side and low-side produces even-order distortion and shifts the operating point of the half-bridge. Excessive dead-time also reduces the amplitude of the reconstructed analog waveform and increases THD.

5 Conclusion

In this practice, we could observe the PWM duty cycle change using each sample of ADC, its too interesting to generate this for some sound amplifiers that require this PWM signal to recreate the sound.